Módulos Integrados del Sistema MONK GRAIN

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

AA1-EV01 explicó cómo se integran los módulos de MONK GRAIN. Esta evidencia es el **expediente de entrega**: dice, pieza por pieza, dónde está cada cosa que la guía pide, qué recibe y devuelve cada módulo, qué pruebas lo cubren y con qué resultado.

Es un documento de ubicación, no de relleno. Donde algo no existe —y hay una cosa importante que no existe— se dice en su lugar en vez de rodearlo.

2. Objetivo

Entregar los módulos integrados del sistema con su documentación: aprobación de requerimientos, repositorio, ejecutables, URL de despliegue, documentación por módulo y componente con sus datos de entrada y salida, informe de pruebas por módulo, configuración de servidores y bases de datos, ambientes de desarrollo y pruebas, y manuales técnicos.

3. Dónde está cada cosa

Lo que pide la guía	Dónde está	Estado
Aprobación de requerimientos	`docs/01-analisis-producto.md`, `docs/02-historias-usuario.md` (531 líneas), y las guías GA1–GA3 de la carpeta del proyecto	✅
Repositorio con el código fuente	github.com/monjejuan448-ux/MONK-GRAIN-APP, rama `main`	✅
Archivos ejecutables	Dos imágenes de Docker, construidas por el CI en cada subida. §5	✅
URL de despliegue	No hay. §9	✅
Documentación por módulo, con entradas y salidas	`MODULOS.md`, junto a este documento. §4	✅
Informe de pruebas por módulo	`MODULOS.md` §Resumen, y §6 de aquí	✅
Configuración de servidores y bases de datos	`docker-compose.yml`, `docker-compose.prod.yml`, `infra/`, `docs/07-despliegue.md` (311 líneas). §7	✅
Ambientes de desarrollo y pruebas	§8, y `docs/07`	✅
Manuales técnicos	Los 18 documentos de `docs/`. §10	✅

4. Documentación por módulo: entradas y salidas

Está en `MODULOS.md`: los **107 endpoints** de los 12 módulos, cada uno con los datos que recibe —parámetros de ruta, de consulta y campos del cuerpo, con su tipo y si es obligatorio — y los que devuelve.

No se escribió a mano, y es una decisión de fondo más que de comodidad. Una tabla de 107 endpoints redactada a mano se desactualiza en la primera semana: alguien añade un campo y no se acuerda del documento. `generar_modulos.py` lo produce del documento OpenAPI que publica el servicio en marcha y del XML de JUnit que produce `pytest`, resolviendo las referencias a los esquemas para que los campos aparezcan en el texto y no como un `$ref` que nadie va a seguir.

Así queda cada endpoint:

```
### `POST /auth/login`

Iniciar sesión.

**Entrada**

| Dato                        | Tipo               |         |
|-----------------------------|--------------------|---------|
| **cuerpo** `LoginRequest` | *application/json* | **ob.** |
|   · email                 | texto              | **ob.** |
|   · password              | texto              | **ob.** |

**Salida** · `200` · `TokenResponse`

| Campo         | Tipo  |
|---------------|-------|
| access_token  | texto |
| refresh_token | texto |
| token_type    | texto |
| expires_at    | texto |
```

Lo único escrito a mano es el párrafo que describe cada módulo y la correspondencia entre la etiqueta de la API —en español— y el paquete del código —en inglés—, que una máquina no puede deducir.

5. Los ejecutables

El sistema no se entrega como un binario sino como **dos imágenes de Docker**, que es la forma en que de verdad se despliega. El trabajo `docker` del CI las construye en cada subida, así que la comprobación de que son construibles ocurre sola:

```
backend/Dockerfile   →  FastAPI + Python 3.12, dependencias fijadas con
                        requirements.lock (78 paquetes)
web/Dockerfile       →  Next.js 15 compilado para producción
```

Para reconstruirlas a mano:

```
$ docker compose -f docker-compose.prod.yml build
$ docker compose -f docker-compose.prod.yml up -d
```

Los módulos Java de GA7 sí producen artefactos convencionales: `catalogo-pergamino-1.0.0.jar`, `catalogo-pergamino.war`, `negociacion-pergamino.jar` y `autenticacion-pergamino.jar`, cada uno con `mvn package`. No se versionan: se regeneran del código, y un JAR de 48 MB en el repositorio no aporta nada.

6. Informe de pruebas por módulo

453 pruebas, 0 fallos, 87 % de cobertura sobre 6.055 líneas. Los números salen del XML de JUnit de la ejecución, no de contar funciones a ojo.

Módulo	Endpoints	Pruebas	Fallos
sistema	1	3	0
autenticación	10	38	0
usuarios	7	36	0
catálogo	5	40	0
carrito	9	29	0
pedidos	6	29	0
devoluciones	10	32	0
reseñas	8	25	0
oferta y demanda	10	40	0
panel del productor	27	63	0
liquidaciones	7	27	0
administración	7	21	0

> **Esa columna suma 383 y las pruebas ejecutadas son 453.** No es un error de cuenta, y `MODULOS.md` lo explica con la resta hecha:
>
> - `test_checkout.py` (18 pruebas) cuenta en **dos** módulos —carrito y pedidos— porque recorre justamente el paso de uno al otro. Sumado dos veces, infla la columna en 18.
> - **88 pruebas no pertenecen a ningún módulo de dominio**: verifican `app/core/` —seguridad, catálogo DANE, archivos, días hábiles— y el mapeo de los modelos.
>
> Cuadra así: `383 − 18 + 88 = 453`.

En el frontend, **53 pruebas** en 5 archivos, más cuatro puertas de calidad que deben pasar todas: `tsc`, `eslint`, `prettier` y las pruebas. Y los módulos Java de GA7 suman **80 pruebas** más: 13 en el de JDBC, 23 en el de negociación con Spring y 44 en el de autenticación. El de servlets y JSP no tiene pruebas automáticas —se verificó por HTTP contra Tomcat—, y es la pieza más débil del conjunto en ese aspecto.

7. Configuración de servidores y bases de datos

Servicio	Versión	Puerto	Para qué
`postgres`	16	5432	54 tablas. La base del sistema
`redis`	7	6379	Topes de frecuencia y caché
`minio`	—	9000 / 9001	Fotos de lotes y evidencias
`mailpit`	—	8025	Correo de desarrollo: nada sale a internet
`backend`	Python 3.12	8000	La API
`web`	Node 22	3000	El sitio
`nginx`	—	80 / 443	**Solo en producción**: TLS y entrada única

7.1 La base de datos

```
PostgreSQL 16 · 54 tablas · 7 migraciones de Alembic

$ docker compose exec backend alembic current
4ad7359adcf8 (head)

$ docker compose exec backend alembic check
No new upgrade operations detected.
```

`infra/postgres/init/01-extensions.sql` crea las extensiones antes de que arranque la aplicación —hace falta para los UUID y la búsqueda de texto— porque una migración no puede crear la extensión que ella misma necesita.

Los módulos Java de GA7 usan una base **distinta**, `monkgrain_java`, en el mismo PostgreSQL. Es deliberado: un módulo de aprendizaje que inserta y borra no debe acercarse a los datos del producto.

7.2 Secretos

Variable	Qué protege
`SECRET_KEY`	La firma de los tokens
`ENCRYPTION_KEY`	El cifrado de los datos bancarios
`POSTGRES_PASSWORD`	La base de datos
`S3_SECRET_KEY`	El almacenamiento de archivos

Ninguna está en el repositorio. `.env.example` lista los nombres con valores de ejemplo; `.env` está excluido de Git, y el CI tiene un paso que **busca secretos en el historial completo** —no solo en el último cambio— porque una clave filtrada hace seis commits sigue filtrada.

8. Ambientes de desarrollo y pruebas

	Desarrollo	Pruebas	Producción
Dónde corre	El equipo del aprendiz	Ejecutores de GitHub	Sin desplegar
Orquestación	`docker-compose.yml`	`.github/workflows/ci.yml`	`docker-compose.prod.yml`
Esquema	`alembic upgrade head`	`create_all` en `conftest.py`	`alembic upgrade head`
Archivos	MinIO	`adobe/s3mock`	S3 o compatible
Datos	Semilla + demostración	Creados y deshechos por prueba	Reales
Correo	Mailpit	—	SMTP real
Recarga en caliente	Sí	—	No

8.1 Aislamiento de las pruebas, y su límite

Cada prueba corre dentro de una transacción que se deshace al terminar. Eso las hace independientes del orden… y comparten la misma base.

> **Consecuencia que cuesta horas:** `pytest` debe ejecutarse **de a uno**. Dos sesiones simultáneas produjeron **326 errores que no existían**. No es un defecto del código; es que el aislamiento es por transacción y no por base. Arreglarlo de verdad exigiría una base por trabajador (`pytest-xdist` con esquemas separados), y se decidió no pagar esa complejidad: la ejecución completa tarda dos minutos y medio.

En el CI el problema no aparece porque cada trabajo levanta su propio PostgreSQL. Curiosamente, **el ambiente de pruebas está mejor aislado que el de desarrollo**.

9. La URL de despliegue: lo que falta

La guía pide «URL donde están desplegados los módulos». **No la hay.** `docker-compose.prod.yml` está escrito, con nginx, TLS y las dos imágenes, y `docs/07-despliegue.md` documenta el procedimiento en 311 líneas, pero **nunca se ha ejecutado contra un servidor**.

Decirlo así es más útil que improvisar un enlace. Y hay una razón por la que este paso no lo puede dar el agente: desplegar implica contratar un servidor, registrar un dominio y poner claves reales de un proveedor de correo y de almacenamiento. Son decisiones con costo y con un responsable, y ese responsable es el aprendiz.

9.1 Qué hace falta, en orden

#	Paso	Qué decidir
1	Un servidor con Docker	2 GB de RAM alcanzan. Una máquina pequeña de cualquier proveedor
2	Un dominio apuntando a su dirección	El nombre
3	Certificado TLS	Let's Encrypt. El procedimiento está en `docs/07` §3
4	Las cuatro claves de §7.2, generadas de nuevo	**No reutilizar las de desarrollo**
5	Un proveedor de correo real	Mailpit no envía nada fuera
6	`alembic upgrade head` y la semilla	Semilla sí, datos de demostración no
7	Respaldos de la base	`docs/07` §5. Sin esto, el despliegue es una demostración, no un servicio

> **Lo que te falta hacer a ti**
>
> **Desplegar el sistema y anotar la URL**, siguiendo los siete pasos de §9.1 y `docs/07-despliegue.md`.
>
> Si la entrega no exige que esté accesible en internet, una alternativa honesta es mostrarlo corriendo en tu equipo con `docker compose up -d` y entregar `http://localhost:3000` con capturas, **diciendo que es local**. Lo que no conviene es entregar un enlace que no responda.
>
> Lo primero es decidir con la instructora si para esta evidencia hace falta un despliegue público o basta el local.

10. Manuales técnicos

18 documentos en `docs/`, escritos a lo largo del proyecto:

Documento	De qué trata
`01-analisis-producto.md`	Qué se construye y para quién
`02-historias-usuario.md`	Las historias con sus criterios de aceptación
`03-arquitectura-y-datos.md`	Capas y modelo de datos
`04-estructura-proyecto.md`	**Los estándares de codificación**
`05-diseno-visual.md`	Paleta, tipografía y componentes
`06-api.md`	La API, en prosa (652 líneas)
`07-despliegue.md`	**Instalación, despliegue y operación** (311 líneas)
`08`, `09`, `17`	Las tres auditorías y lo que se corrigió
`10-revision-legal.md`	Contraste con la normativa colombiana
`11` a `16`, `18`	Devoluciones, reseñas, comprobante, pergamino, oferta y demanda, liquidaciones

11. Verificación de la entrega

```
$ docker compose ps                          6 servicios running
$ curl -s localhost:8000/health              {"status":"ok"}
$ curl -s localhost:8000/openapi.json        107 endpoints
$ alembic current                            4ad7359adcf8 (head)
$ alembic check                              No new upgrade operations detected
$ pytest -q --cov=app                        453 passed · 87 %
$ npm test                                   53 passed
$ newman run (GA7-AA5-EV04)                  28 peticiones · 48 comprobaciones
```

12. Conclusión

El expediente está completo salvo una cosa, y esa cosa está dicha en §9 con nombre propio: **no hay URL de despliegue**, porque desplegar exige un servidor, un dominio y claves reales que solo el aprendiz puede contratar y asumir.

De todo lo que se entrega, lo que más vale a medio plazo no es ninguno de los documentos sino `generar_modulos.py`. Las entradas y salidas de 107 endpoints es exactamente el tipo de

documentación que se escribe una vez, se desactualiza en una semana y a partir de entonces estorba más que ayuda. Generada del servicio en marcha, no puede decir una cosa distinta del código: si alguien añade un campo, el documento lo trae la próxima vez que se ejecute.